

Documentação de Requisitos - ChainsawBot

Visão Geral do Sistema

O sistema ChainsawBot é uma aplicação de automação composta por um web scraper e um bot de mensageria. Seu objetivo é monitorar o Ambiente Virtual de Aprendizagem (AVA - Canvas PUCPR), extrair novas tarefas acadêmicas, armazená-las localmente para evitar duplicidade de notificações, e distribuir arquivos de calendário (`.ics`) para os alunos cadastrados via Telegram. A aplicação será containerizada para garantir execução ininterrupta e estabilidade de dependências.

Requisitos Funcionais (RF)

Ações que o sistema deve ser capaz de realizar.

- **RF01 - Autenticação Automática:** O sistema deve realizar login automático no Canvas PUCPR utilizando credenciais válidas de aluno.
 - **RF02 - Varredura de Disciplinas:** O sistema deve navegar por todas as abas de matérias/cursos ativas no perfil logado.
 - **RF03 - Extração de Tarefas:** O sistema deve ler e extrair os dados da seção “Tarefas Futuras” de cada matéria.
 - **RF04 - Exceções de Cursos:** O sistema não deve buscar pelos cursos que não contem a aba de tarefas.
 - **RF05 - Armazenamento de Dados:** O sistema deve salvar as tarefas extraídas em um banco de dados local (JSON ou NoSQL).
 - **RF06 - Gestão de Usuários (Telegram):** O sistema deve registrar o `chat_id` dos alunos que iniciarem uma conversa com o bot (ex: comando `/start`), para saber para quem enviar os arquivos.
 - **RF07 - Geração de Calendário:** O sistema deve criar um arquivo `.ics` contendo as informações das tarefas extraídas.
 - **RF08 - Disparo de Mensagens:** O sistema deve enviar o arquivo `.ics` via Telegram de forma passiva (push) para todos os usuários com o bot ativo na conversa.
 - **RF09 - Requisição Manual de Calendário:** O sistema deve permitir que o utilizador solicite o último ficheiro `.ics` gerado através de um comando no Telegram (ex: `/calendario`), enviando-o imediatamente em resposta.
-

Regras de Negócio (RN)

Lógicas, condições e restrições que guiam as funcionalidades.

- **RN01 - Agendamento Diário:** A rotina de varredura (scraping) e notificação deve ser executada obrigatoriamente uma vez por dia, às **00:00**.
 - **RN02 - Estrutura de Dados da Tarefa:** Toda tarefa salva no banco deve conter, no mínimo: Data de Entrega, Tipo da Tarefa, Descrição e Matéria.
 - **RN03 - Mapeamento de Tipos:** O sistema deve categorizar as tarefas conforme a nomenclatura acadêmica (Ex: Atividade em sala, Tarefa, Formativa, Avaliação Somativa, Lista de Exercícios, TDE).
 - **RN04 - Exclusão de Tarefas Inválidas:** O scraper deve ignorar e descartar qualquer tarefa que não possua uma data de entrega vinculada.
 - **RN05 - Ciclo de Vida da Tarefa (Status):** O banco de dados deve classificar cada tarefa em três estados:
 - *Não enviada:* Tarefa recém-coletada, pendente de notificação.
 - *Já enviada:* Tarefa cujo `.ics` já foi gerado e enviado aos alunos.
 - *Expirada:* Tarefa cuja data de entrega já passou.
 - **RN06 - Condição de Geração do ICS:** O arquivo `.ics` deve ser populado **exclusivamente** com tarefas no status *"Não enviada"*.
 - **RN07 - Condição de Envio:** O sistema não deve gerar arquivo nem disparar mensagens no Telegram em dias onde não houver nenhuma tarefa com status *"Não enviada"*. Após o envio bem-sucedido, o status destas tarefas deve ser atualizado para *"Já enviada"*.
 - **RN08 - Comportamento na Requisição Manual:** Ao receber o comando manual, o bot deve enviar a última versão do ficheiro gerada localmente, ou notificar o utilizador com uma mensagem de erro caso ainda não exista nenhum ficheiro `.ics` disponível no sistema.
-

Requisitos Não Funcionais (RNF)

Especificações técnicas, arquitetura e restrições de ambiente.

- **RNF01 - Stack de Scraping:** A extração de dados deve ser feita em Python, operando o navegador em modo *headless* (sem interface gráfica visível), utilizando a biblioteca Playwright.
 - **RNF02 - Persistência Local:** O armazenamento deve ser implementado de forma leve. Recomenda-se o uso de arquivos estruturados em `JSON` ou bancos NoSQL orientados a documentos adequados para Python (como o `TinyDB`).
 - **RNF03 - Integração Telegram:** A comunicação com o bot deve utilizar a API oficial do Telegram via biblioteca `pyTelegramBotAPI`.
 - **RNF04 - Job Scheduler (Agendador):** O gatilho diário das 00:00 deve ser controlado por um agendador de tarefas em código (como a biblioteca `schedule` no Python) ou pelo agendador nativo do sistema operacional.
 - **RNF05 - Containerização (Docker):** A aplicação deve ser encapsulada em um contêiner Docker. Isso garante a padronização do ambiente (resolvendo dependências gráficas do Playwright no Linux) e permite a execução estável em segundo plano (24/7) em servidores na nuvem (VPS).
-

Estrutura de Módulos (Arquitetura)

Para garantir a organização e facilidade de manutenção, o sistema será dividido nos seguintes módulos:

1. **scraper.py** : Responsável pela automação com Playwright, login no AVA e retorno da lista de tarefas extraídas.
2. **database.py** : Gerencia a leitura e escrita no arquivo JSON/TinyDB, compara tarefas existentes, cadastra tarefas novas como "*não enviada*" e atualiza o status das antigas.
3. **calendar_gen.py** : Consome os dados do banco filtrando por tarefas "*não enviadas*", formata os dados com a biblioteca **ics** e exporta o arquivo.
4. **bot.py** : Processo independente que escuta interações dos usuários no Telegram, registra os novos IDs no banco e processa requisições manuais do usuário (comando **/calendario**).
5. **main.py** : O orquestrador da aplicação. Contém o agendador (**schedule**) que roda diariamente à meia-noite, chamando o scraper, atualizando o banco, gerando o calendário e acionando o envio em massa via Telegram.
6. **Dockerfile** : Arquivo de configuração para construção da imagem do sistema, contendo as instruções do sistema operacional base, dependências e comando de inicialização.